

coreReports requirements**1. General Application Concept**

The system is a comprehensive platform designed to manage, assign, document, and analyze maintenance operations (preventive and corrective) and technical services. It allows companies (departments) to maintain precise control over requests from their clients or internal departments, assign technicians to these tasks, and generate detailed technical reports that serve as work receipts and material logs.

2. Base Architecture (Multi-Tenant)

The entire application is built on a multi-tenant architecture.

- The core component is the Departments table/entity.
- Golden Rule: Every record in the system (Users, Tickets, Services, Reports) is strictly tied to a Department ID. This ensures that users of "Company A" can never see the information of "Company B," allowing the software to be sold under a Software as a Service (SaaS) licensing model.

3. User Roles and Permissions

The system manages four hierarchical access levels (roles), each with a clear operational purpose:

- Role 0 (Default/Client): Can only create tickets (request help) and view the status of their own requests.
- Role 3 (Maintenance Technician): Field personnel. They receive work assignments, perform tasks, and are primarily responsible for completing "Technical Reports," documenting hours, materials, and solutions.
- Role 1 (Technical Administrator): The operations supervisor. Can reassign tickets, view all technicians in their department, and acts as the "Responsible Party" who approves the generated reports.
- Role 2 (Commercial Administrator): Focused on managing the department (billing, license control, business KPIs).

4. Main Operational Flow (The Lifecycle)

The system divides the work into two main branches that converge at the same destination (The Report):

1. Reactive Maintenance (Tickets): Something breaks. A user creates a Ticket (e.g., "Hoist Failure"). It enters with an Open status.
2. Proactive Maintenance (Scheduled Services): Jobs planned in advance (e.g., "Annual Maintenance," "Network Installation"). They have strict due dates.

coreReports requirements

The Flow:

- Assignment: An Administrator assigns the Ticket or Service to a Technician.
- Execution and Documentation: The Technician performs the work and opens the Reports Module.
- Automatic Closure (State Machine): If the technician marks the report as "Job Completed," the system automatically changes the status of the parent Ticket to Resolved (or the Service to Completed). If they need to return tomorrow, they mark it as "Partial Work," and the Ticket remains in In Progress.

5. The Reporting Module (The System Engine)

This is the most robust and important module of the application. A single ticket/service can generate multiple reports (e.g., a job that takes 3 days will generate 3 reports).

Key technical features for the backend:

- Polymorphism: A report can originate from a ticket, a service, or be "free" (without a known origin).
- Many-to-Many Relationships (Executors): Supports multiple technicians working on the same report.
- Basic Inventory Control (Materials): Dynamically records each part, cable, or spare part used in the job using a relational table (materials_report).
- Time Calculation: Requires a start and end date/time, allowing the system to mathematically calculate the actual duration of the work in the field without the user having to perform manual calculations.

6. KPI and Analytics Engine (Dashboard)

Thanks to the strict normalization of the relational database, the system does not require static statistics tables. The dashboard will use real-time SQL queries to display:

- Total hours worked (overall or per technician).
- Service performance (Services completed on time vs. overdue).
- Operational volume (Number of tickets opened vs. reports generated).

coreReports requirements**Architecture Style:** Relational (SQL)

Objective: Update the database schema to support the new UI-driven frontend architecture, normalizing complexing arrays (Executors, Material, Photos) and establishing solid Foreign Key constraints for Multi-Tenant capabilities

1. Multi-Tenant & Access TablesTable: *dependencias*

Core table for Multi-Tenant architecture

Column	Type	Constraints / Notes
idDependencia	INT	PK , Auto-increment
nombre	VARCHAR(100)	Not Null
rfc	VARCHAR(20)	
tipoLicencia	ENUM	'Prueba gratuita', 'Mensual', 'Anual'
fechaExpiracionLicencia	DATE	
correoContacto	VARCHAR(100)	
correoReportes	VARCHAR(100)	
telefono	VARCHAR(20)	
datosFacturacion	TEXT	
activo	BOOLEAN/TINYINT	Default 1 (True)
fechaRegistro	DATETIME	

coreReports requirementsTable: *usuarios*

Column	Type	Constraints / Notes
idUsuario	INT	PK , Auto-increment
idDependencia	INT	FK -> dependencias.idDependencia
numeroEmpleado	VARCHAR(45)	
rol	INT	0: Default, 1: Admin Tec, 2: Admin Com, 3: Tech
nombre	VARCHAR(100)	Not Null
apellidos	VARCHAR(100)	Not Null
correo	VARCHAR(100)	Unique
password	VARCHAR(255)	Hashed
estado	TINYINT	Default 1 (Active)
fechaCreacion	DATETIME	
ultimoAcceso	DATETIME	

coreReports requirements**Origin Data Tables (Tickets & Services)**Table: *tickets*

Column	Type	Constraints / Notes
idTicket	INT	PK , Auto-increment
idDependencia	INT	FK -> dependencias.idDependencia
folio	VARCHAR(50)	Unique per Dependencia
asunto	VARCHAR(200)	Not Null
descripcion	TEXT	
solicitante	VARCHAR(100)	
cargo	VARCHAR(100)	
prioridad	ENUM	'Alta', 'Media', 'Baja'
estatus	ENUM	'Abierto', 'En Proceso', 'Cerrado', 'Resuelto'
fechaSolicitud	DATETIME	
fechaAtencion	DATETIME	Nullable
idUsuarioAtendio	INT	FK -> usuarios.idUsuario (Replaces string)

coreReports requirementsTable: *servicios*

Column	Type	Constraints / Notes
idServicio	INT	PK , Auto-increment
idDependencia	INT	FK -> dependencias.idDependencia
folio	VARCHAR(50)	Unique per Dependencia
asunto	VARCHAR(200)	Not Null
descripcion	TEXT	
solicitante	VARCHAR(100)	
cargo	VARCHAR(100)	
categoria	ENUM	'Mantenimiento Preventivo', 'Mantenimiento Correctivo', 'Instalación', 'Mejora'
idUsuarioAsignado	INT	FK -> usuarios.idUsuario (Nullable)
fechaAsignacion	DATE	
fechaVencimiento	DATE	
estatus	ENUM	'Programado', 'En Proceso', 'Realizado', 'Vencido'
prioridad	VARCHAR(50)	

coreReports requirements**Reports Module (Core & Normalized Sub-tables)**Table: *reportes*

Column	Type	Constraints / Notes
idReporte	INT	PK , Auto-increment
idDependencia	INT	FK -> dependencias.idDependencia
folio	VARCHAR(50)	Unique per Dependencia
creadoPor	INT	FK -> usuarios.idUsuario
tipoReporte	ENUM	'Ticket', 'Servicio Programado', 'Libre'
origenDatosId	INT	Nullable. <i>Note: Polymorphic ID (Matches idTicket or idServicio depending on tipoReporte).</i>
fechaElaboracion	DATETIME	
responsableId	INT	FK -> usuarios.idUsuario
fechaHoraInicio	DATETIME	
fechaHoraFin	DATETIME	
desarrolloActividades	TEXT	
estatusReporte	ENUM	'Abierto / Trabajo Parcial', 'Cerrado / Trabajo Finalizado'

coreReports requirements

Table: *reporte_ejecutores* (N:M relationship)

Normalizes the **ejecutoresIds** array (this will be a “front-problem”)

Column	Type	Constraints / Notes
idRegistro	INT	PK , Auto-increment
idReporte	INT	FK -> reportes.idReporte (Cascade Delete)
idUsuario	INT	FK -> usuarios.idUsuario (The Technician)

Table: *reporte_materiales* (1: N Relationship)

Normalizes the **materiales** array

Column	Type	Constraints / Notes
idMaterial	INT	PK , Auto-increment
idReporte	INT	FK -> reportes.idReporte (Cascade Delete)
idItem	INT	Order indicator (1, 2, 3...)
material	VARCHAR(150)	Not Null
cantidad	DECIMAL(10,2)	Allows decimals for things like Liters/Meters
unidad	VARCHAR(50)	'Pza', 'Metros', 'Litros', etc.

coreReports requirements

Table: *reporte_evidencias* (1:N Relationship)

Normalizes the *evidenciaFotografica* array

Column	Type	Constraints / Notes
idEvidencia	INT	PK , Auto-increment
idReporte	INT	FK -> reportes.idReporte (Cascade Delete)
urlFotografia	VARCHAR(255)	Server/S3 path to the image

KPI & Metrics Guide (No extra tables, thought xd)

Dev Note: The frontend requires the following metrics. Please implement these as SQL queries/Views rather than storing them statically.

- Total_horas: SUM(TIMESTAMPDIFF(HOUR, fechaHoraInicio, fechaHoraFin)) from reportes.
- Total_horas_tecnico: Join reportes with reporte_ejecutores on idReporte, filter by idUsuario, and SUM the time difference.
- Cantidad_reportes_generados: COUNT(idReporte) from reportes.
- Cantidad_servicios_programados: COUNT(idServicio) from servicios.
- Cantidad_tickets_levantados: COUNT(idTicket) from tickets.
- Cantidad_sp_vencidos: COUNT(idServicio) from servicios WHERE estatus = 'Vencido' (or dynamic fechaVencimiento < CURDATE()).
- Cantidad_sp_realizados_en_tiempo: COUNT(idServicio) from servicios WHERE estatus = 'Realizado' AND (assuming fechaAtencion logic or dynamic status updates before fechaVencimiento).